

Softwarekonstruktion WS 16/17 – Übung 5

Ausgabe: 19.12.2016, Abgabe: 16.01.2017 12:00 Uhr

Präsenzübung

5.1 White-Box-Testen

5.1.1 Kontrollflussbezogenes Testen – Testfälle

Das gegebene Programmsegment soll mit einem Kontrollfluss-Testverfahren getestet werden:

```
1 read (x,y,z)
2 if ( (x>0) AND (y>0) AND (z>0) ) { x:=x*y*z }
3 else { x:=y }
4 if ( x+y+z>0 ) { z:=-x }
```

1. Geben Sie eine minimale Testmenge für eine Anweisungsüberdeckung an.

Lösungsvorschlag

$\{(1,1,1);(-1,-1,-1)\}$

2. Geben Sie eine minimale Testmenge für eine einfache Bedingungsabdeckung an.

Lösungsvorschlag

$\{(1,1,-1);(-1,-1,1)\}$

3. Geben Sie eine minimale Testmenge für eine Mehrfachbedingungsabdeckung an.

Lösungsvorschlag

$\{(1,1,1);(1,1,-1);(1,-1,1);(1,-1,-1);(-1,1,1);(-1,1,-1);(-1,-1,1);(-1,-1,-1)\}$

4. Geben Sie eine minimale Testmenge für eine minimal bestimmende Mehrfachbedingungsabdeckung an.

Lösungsvorschlag

$\{(1,1,1);(1,1,-1);(1,-3,1);(-1,1,1)\}$

5. Geben Sie eine minimale Testmenge für eine Pfadabdeckung an.

Lösungsvorschlag

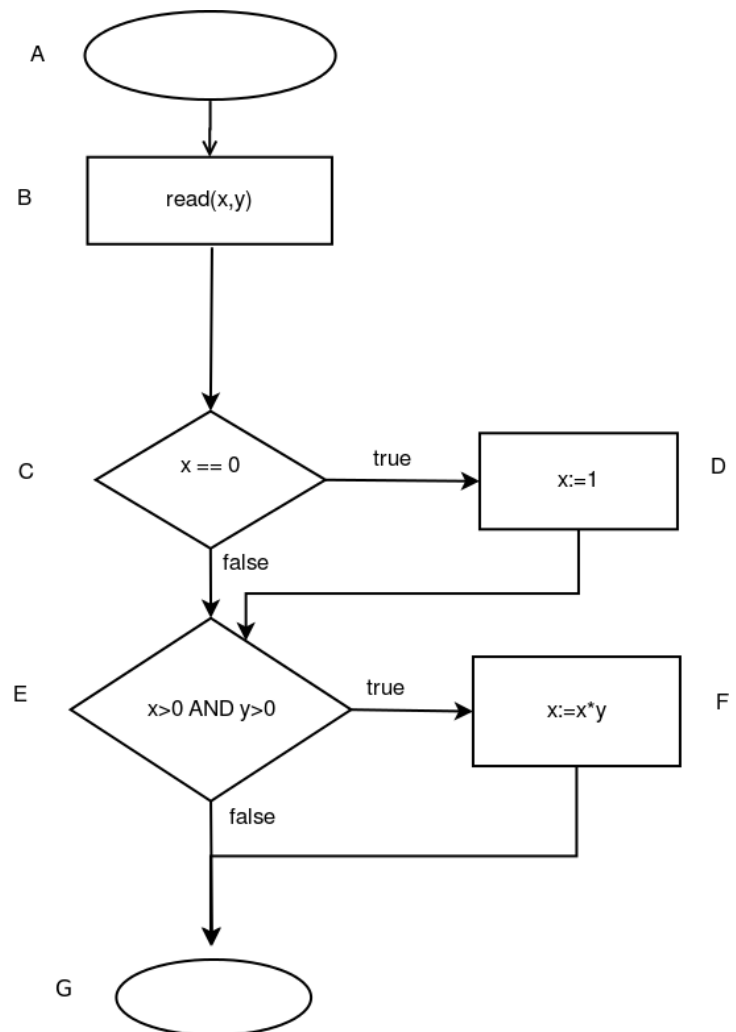
Es gibt keine Pfadabdeckung, die die Bedingungen “ $(x > 0) \text{ AND } (y > 0) \text{ AND } (z > 0)$ ” und “ $(x+y+z > 0)$ ” voneinander abhängig sind und deswegen $x + y + z > 0$ immer gilt, wenn $((x > 0) \text{ AND } (y > 0) \text{ AND } (z > 0))$ gilt.

5.1.2 Kontrollflussbezogenes Testen – Testerfüllung

Das gegebene Programmsegment soll getestet werden:

```
1 read(x,y)
2 if ( x==0 ) { x:= 1 }
3 if ( (x>0) AND (y>0) ) { x:=x*y }
```

Nehmen Sie für die Aufgaben 3-7 an, es würde ein Test mit den Testdaten $(x=0, y=1)$ und $(x=-1, y=5)$ durchgeführt.



1. Skizzieren Sie das Flussdiagramm.
2. Geben Sie alle Entscheidungswege an.

Lösungsvorschlag

$\{A, B, C\}; \{C, D, E\}; \{C, E\}; \{E, G\}; \{E, F, G\}$

3. Erfüllt diese Testmenge die Anweisungsüberdeckung? Begründen Sie Ihre Antwort.

Lösungsvorschlag

Dies Testmenge erfüllt die Anweisungsüberdeckung. Mit dem Test $(x=0, y=1)$ werden alle Anweisungen durchlaufen.

4. Erfüllt diese Testmenge die Zweigüberdeckung? Begründen Sie Ihre Antwort.

Lösungsvorschlag

Die Testmenge erfüllt die Zweigüberdeckung. Der erste Test durchläuft die jeweiligen 'true'-Zweige, der zweite Test durchläuft die entsprechenden 'false'-Zweige, so dass alle Zweige durchlaufen werden.

5. Erfüllt diese Testmenge die Entscheidungsüberdeckung? Begründen Sie Ihre Antwort.

Lösungsvorschlag

Die Testmenge erfüllt die Entscheidungsüberdeckung. Der erste Test durchläuft die jeweiligen 'true'-Zweige, der zweite Test durchläuft die entsprechenden 'false'-Zweige, so dass alle Entscheidungen einmal 'true' und einmal 'false' sind.

6. Erläutern Sie den Unterschied zwischen einer Entscheidungs- und einer Zweigüberdeckung. Was ändert sich, wenn man den Abbruch von Tests mit einbezieht?

Lösungsvorschlag

Bei Betrachtung vollständiger Testpfade kommen beide zu demselben Ergebnis. Betrachtet man bei unvollständigen Testpfaden den Überdeckungsgrad, so haben beide eine unterschiedliche Metrik.

7. Erfüllt diese Testmenge die einfache Bedingungsüberdeckung? Begründen Sie Ihre Antwort.

Lösungsvorschlag

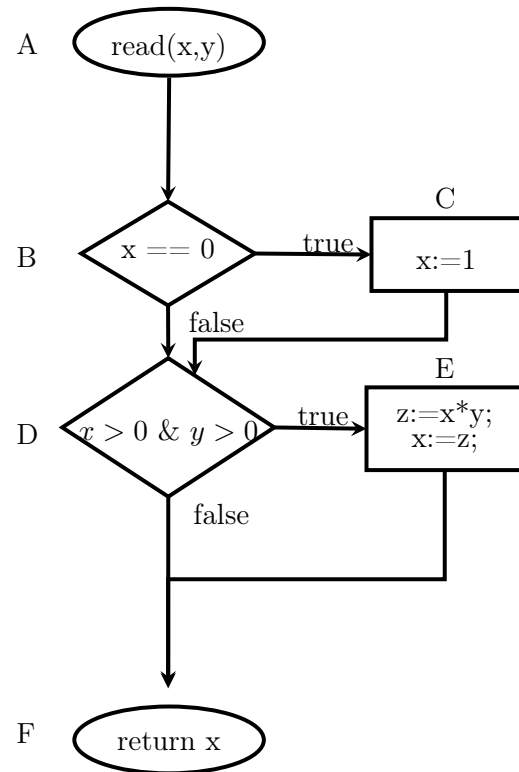
Die Testmenge erfüllt die einfache Bedingungsüberdeckung nicht, da das atomare Prädikat $y > 0$ nie zu false ausgewertet wird.

5.1.3 Datenflussbezogenes Testen

Gegeben sei das Programmsegment:

```
1. read(x,y)
2. if ( x==0 ) { x:= 1 }
3. if ( (x>0) AND (y>0) )
   { z:=x*y; x:=z;}
4. return x;
```

und das zugehörige Flussdiagramm:



1. Geben Sie für jeden der Knoten k im Kontrollflußdiagramm die Mengen $DEF(k)$, $UNDEF(k)$ und $REF(k)$ an.

Lösungsvorschlag

	A	B	C	D	E	F
$DEF(k)$	{x,y}	{ }	{x}	{ }	{z,x}	{ }
$UNDEF(k)$	{ }	{ }	{ }	{ }	{ }	{ }
$REF(k)$	{ }	{x}	{ }	{x,y}	{x,y,z}	{x}

2. Was fällt bei Knoten E auf? Welche Konsequenz ergibt sich daraus?

Lösungsvorschlag

Es liegt ein rein lokaler Datenfluss vor: z wird erst definiert und dann referenziert. Der Knoten muss deswegen in zwei Knoten aufgeteilt werden: $E_1 : z := x*y$ und $E_2 : x := z$. Daraus ergeben sich dann $DEF(E_1)=\{z\}$, $REF(E_1)=\{x,y\}$ und $DEF(E_2)=\{x\}$, $REF(E_2)=\{z\}$.

3. Gegeben seien nun die Testdaten: $\{x=0, y=1\}$. Bestimmen Sie welche der folgenden Kriterien von Datenflüssen erfüllt sind und begründen Sie Ihre Aussage.

Alle Definitionen

Alle DR-Interaktionen

Alle Referenzen

Lösungsvorschlag

Alle Definitionen Ja, alle Zuweisungen werden auch mindestens einmal verwendet.

Alle DR-Interaktionen Nein, vom Knoten A wird mit den Testdaten Knoten D nicht erreicht. Da dieser aber über den Knoten B erreichbar wäre, erfüllen die Testdaten die Voraussetzungen für alle DR-Interaktionen nicht.

Alle Referenzen Nein, es werden nicht jeweils alle Zweige nach den Entscheidungsknoten durchlaufen. z.B. nicht $B \rightarrow D$.

Hausübung

5.2 White-Box-Testen (20 Punkte)

Hinweis zu dieser Hausübung: Diese Aufgabe muss von jedem Teilnehmer in Moodle durchgeführt und abgeschlossen werden, kann jedoch in Gruppen besprochen werden. Die Bearbeitung *dieser* Aufgabe ist möglich bis zum 16. Januar 2017 12:00 Uhr.

Die übrigen Aufgaben müssen auf Papier in den Briefkasten geworfen werden (Gruppenabgabe: bis zu vier Personen einer Übungsgruppe).

5.3 Konfigurationsmanagement (2 Punkte)

5.3.1 Versionskontrolle

Beschreiben Sie das *Pessimistische* Modell der Versionskontrolle:

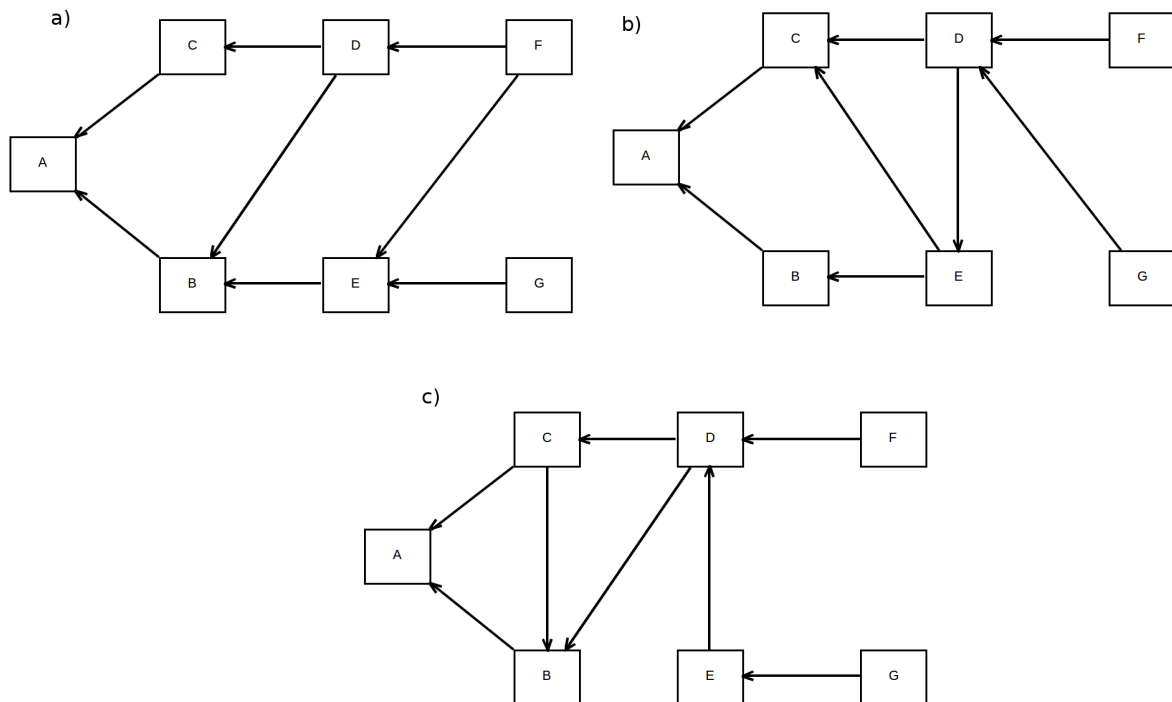
1. Beschreiben Sie die einzelnen Schritte, wenn 2 Personen eine Datei bearbeiten wollen.
2. Welche Probleme können bei dem *Pessimistischem* Modell im Allgemeinen auftreten?

1. Beschreiben Sie die einzelnen Schritte, wenn 2 Personen eine Datei bearbeiten wollen:
 - a) Person A sperrt Datei
 - b) Person A bearbeitet Datei
 - c) Person A gibt Datei frei
 - d) Person B sperrt Datei
 - e) Person B bearbeitet Datei
 - f) Person B entsperrt Datei
2. Welche Probleme können bei dem *Pessimistischem* Modell im Allgemeinen auftreten?
 - a) Entsperren kann vergessen werden.
 - b) Paralleles Arbeiten an einer Datei nicht möglich
 - c) Deadlock ist möglich.

5.4 Buildmanagement (8 Punkte)

5.4.1 Erstellungsstrategien

Betrachten Sie folgende Abhängigkeitsgraphen für die Module *A* bis *G*:

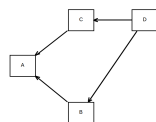


1. Geben Sie jeweils alle möglichen Reihenfolgen der Module an, wenn Modul *A* mittels rekursivem Make kompiliert wird.
2. Zeichnen Sie jeweils den partiellen DAG für den Beta-Algorithmus, wenn das Modul *D* aktualisiert wurde.

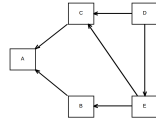
Lösungsvorschlag

Loesungen z.B.

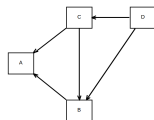
1. a) FGEDBCA, FDGEBBCA, FDCGEBA, GFEDBCA
b) GFDECBA, GFDEBCA, FGDECBA, FGDEBCA
c) GEFDCBA, FGEDCBA



2. a)



b)



c)